

# Práctica 2: Técnicas de Búsqueda basadas en Poblaciones para el Problema de Portfolio

Andrés Borja

Curso académico 2025-2026

# Índice

|                                                                                                               |           |
|---------------------------------------------------------------------------------------------------------------|-----------|
| <b>1. Descripción del problema</b>                                                                            | <b>3</b>  |
| <b>2. Aplicación de los algoritmos</b>                                                                        | <b>4</b>  |
| 2.1. Representación de las Soluciones . . . . .                                                               | 4         |
| 2.2. Retorno . . . . .                                                                                        | 4         |
| 2.3. Fórmulas . . . . .                                                                                       | 4         |
| 2.4. Función Objetivo . . . . .                                                                               | 4         |
| 2.5. Operadores Comunes . . . . .                                                                             | 4         |
| 2.5.1. Operador de selección: Torneo . . . . .                                                                | 4         |
| 2.6. Operadores de cruce . . . . .                                                                            | 5         |
| 2.6.1. Aritmético . . . . .                                                                                   | 5         |
| 2.6.2. $BLX - \alpha$ . . . . .                                                                               | 6         |
| 2.7. Operadores de mutación . . . . .                                                                         | 6         |
| 2.8. Actualizar Fitness . . . . .                                                                             | 7         |
| <b>3. Descripción de los algoritmos</b>                                                                       | <b>8</b>  |
| 3.1. Algoritmos Genéticos . . . . .                                                                           | 8         |
| 3.1.1. AGG-ARIT: Algoritmo Genético Generacional (con elitismo) con operador de cruce aritmético . . . . .    | 8         |
| 3.1.2. AGG-BLX: Algoritmo Genético Generacional (con elitismo) con operador de cruce $BLX - \alpha$ . . . . . | 9         |
| 3.1.3. AGE-ARIT: Algoritmo Genético Estacionario con operador de cruce aritmético . . . . .                   | 10        |
| 3.1.4. AGE-BLX: Algoritmo Genético Estacionario con operador de cruce $BLX - \alpha$ . . . . .                | 11        |
| 3.2. Algoritmos Meméticos . . . . .                                                                           | 12        |
| 3.2.1. AM-All . . . . .                                                                                       | 12        |
| 3.2.2. AM-Rand . . . . .                                                                                      | 14        |
| 3.2.3. AM-Best . . . . .                                                                                      | 16        |
| 3.2.4. Uso de Búsqueda Local (BL) . . . . .                                                                   | 18        |
| <b>4. Estructura del código y manual de usuario</b>                                                           | <b>20</b> |
| 4.1. Estructura del código . . . . .                                                                          | 20        |
| 4.2. Compilación . . . . .                                                                                    | 20        |
| 4.3. Ejecución . . . . .                                                                                      | 20        |
| 4.4. Gráficas . . . . .                                                                                       | 20        |
| <b>5. Experimentos y análisis de resultados</b>                                                               | <b>21</b> |
| 5.1. Descripción de los experimentos . . . . .                                                                | 21        |
| 5.2. Resultados . . . . .                                                                                     | 21        |
| 5.3. Análisis de resultados . . . . .                                                                         | 23        |
| <b>6. Algoritmos extra</b>                                                                                    | <b>25</b> |
| 6.1. Extra 1: AM-BEST-MOD . . . . .                                                                           | 25        |
| 6.2. Extra 2: AGE-BLX-MOD . . . . .                                                                           | 25        |
| <b>7. Referencias</b>                                                                                         | <b>26</b> |

# 1. Descripción del problema

El problema del portfolio consiste en encontrar la asignación óptima de capital entre un conjunto de  $N$  activos financieros disponibles, de forma que se maximice el beneficio y se minimice el riesgo.

Cada solución del problema se representa mediante un vector de pesos  $w = (w_1, w_2, \dots, w_N)$ , donde  $w_i$  indica la proporción de capital invertida en el activo  $i$ . Deberán de cumplir ciertas restricciones que ahora veremos.

El beneficio asociado a una solución se calcula a partir de los retornos históricos de los activos, mientras que el riesgo se modela mediante la matriz de covarianzas, que recoge la dependencia entre los distintos activos. De esta forma, el problema plantea un compromiso entre maximizar la rentabilidad esperada y minimizar la variabilidad de dicha rentabilidad.

Este problema lo vamos a abordar mediante el uso de técnicas metaheurísticas.

## 2. Aplicación de los algoritmos

### 2.1. Representación de las Soluciones

Para representar una solución se usará un vector de pesos reales  $w = (w_1, w_2, \dots, w_n)$  donde cada elemento  $w_i$  representa la proporción del capital total que se invertirá en la empresa  $i$ . Las restricciones para que una solución sea válida son las siguientes:

- **Suma de pesos igual a 1:**  $\sum_{i=1}^n w_i \approx 1$ . Se pone  $\approx$  porque con valores reales las comparaciones se hacen con un cierto margen de error, como  $10^{-8}$ .
- **Rango limitado:**  $(w_i = 0) \vee (lo \leq w_i \leq hi), \forall i \in 1..N$ . Los valores  $lo$  y  $hi$  se definirán más adelante según los datos con los que trabajemos.

### 2.2. Retorno

Para cada día  $t$  del periodo y en cada empresa  $i$  tenemos el retorno obtenido  $\mu_{ti}$  que viene dado por:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Un valor positivo indica que ha habido mejora, y un valor negativo indica que se ha perdido dinero ese día.

### 2.3. Fórmulas

A partir de la matriz de retornos podremos calcular:

| Concepto         | Para qué se usa                                                           | Fórmula                                                                                          |
|------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Media por acción | Retorno medio de una empresa en el periodo $T$ .                          | $\bar{\mu}_i = \left( \sum_{t=1}^T \mu_{ti} \right) / T$                                         |
| Covarianza       | Medir cómo varían conjuntamente dos acciones para evaluar su dependencia. | $\Sigma_{ij} = \frac{1}{T} \sum_{t=1}^T (\mu_{ti} - \bar{\mu}_i) \cdot (\mu_{tj} - \bar{\mu}_j)$ |
| Beneficio        | Calcular la ganancia total esperada de la cartera usando logaritmos.      | $Beneficio(w) = \sum_i w_i \cdot \left( \sum_{t=1}^T \log(1 + \mu_{ti}) \right)$                 |
| Riesgo           | Cuantificar la volatilidad o peligro de pérdida total de la cartera.      | $Riesgo(w) = \sqrt{\sum_i \sum_j w_i \cdot \Sigma_{ij} \cdot w_j}$                               |

Cuadro 1: Métricas

### 2.4. Función Objetivo

Función objetivo a maximizar: equilibra el beneficio obtenido penalizando el riesgo

$$fitness(w) = Beneficio(w) - \lambda \cdot Riesgo(w)$$

Donde usaremos un valor de  $\lambda = 500$

El pseudocódigo es similar al de la practica 1, solo que en el riesgo devolvemos la raíz de lo que antes devolvíamos.

### 2.5. Operadores Comunes

En cuanto a los operadores comunes seguimos usando los de la práctica 1, y además añadimos algunos más también.

#### 2.5.1. Operador de selección: Torneo

Tenemos un torneo para los genéticos generacionales y otro para los genéticos estacionarios. En el primero se seleccionará una población de padres del mismo tamaño que la población genética mientras que en el segundo se seleccionarán únicamente dos padres en cada iteración. Ambos usaran  $k = 3$  lo que significa que para elegir un padre en cada iteración se seleccionan 3 de la población y nos quedamos con el mejor de esos 3.

Para los generacionales rellenamos el vector de padres directamente y copiamos también sus fitness:

```

1 funcion seleccionTorneo(poblacion, fitness_poblacion, referencia padres, referencia
  fitness_padres):
2   para i <- 0 hasta TAM_POBLACION - 1 hacer
3     indice_ganador <- aleatorio(0, TAM_POBLACION - 1)
4     para j <- 1 hasta K_TORNEO - 1 hacer
5       indice_rival <- aleatorio(0, TAM_POBLACION - 1)
6       si fitness_poblacion[indice_rival] > fitness_poblacion[indice_ganador]
7         entonces
8           indice_ganador <- indice_rival
9       fin si
10    fin para
11    padres[i] <- poblacion[indice_ganador]
12    fitness_padres[i] <- fitness_poblacion[indice_ganador]
13  fin funcion

```

Para los estacionarios solo devolvemos un padre, devolvemos su indice en la población:

```

1 funcion seleccionTorneo(poblacion, fitness_poblacion):
2   indice_ganador <- aleatorio(0, TAM_POBLACION - 1)
3   para j <- 1 hasta K_TORNEO - 1 hacer
4     indice_rival <- aleatorio(0, TAM_POBLACION - 1)
5     si fitness_poblacion[indice_rival] > fitness_poblacion[indice_ganador] entonces
6       indice_ganador <- indice_rival
7   fin si
8   fin para
9   retornar indice_ganador
10 fin funcion

```

## 2.6. Operadores de cruce

### 2.6.1. Aritmético

Para el cruce aritmético, lo que hacemos es en cada empresa nos quedamos con un porcentaje  $r \in [0, 1]$  de un padre y  $1 - r$  del otro y para el otro hijo los porcentajes al revés. Es decir si  $S_1$  y  $S_2$  son dos soluciones padre, entonces los dos hijos en cada empresa  $i$  son de la forma:  $\text{Off}_1 = r_i \cdot S_{i1} + (1 - r_i) \cdot S_{i2}$ ,  $\text{Off}_2 = r_i \cdot S_{i2} + (1 - r_i) \cdot S_{i1}$ . Después de este cruce es importante reparar la solución porque seguramente estemos fuera de los rangos y tampoco sume 1 la suma de los pesos. El cruce es el mismo en generacionales que en estacionarios. Voy a mostrar como lo he hecho para generacionales de forma que rellena el vector de hijos directamente, teniendo en cuenta que solo se cruzan los primeros  $\text{NUM\_CRUCES\_ESPERADO}$ , que mantiene aleatoriedad por la forma de seleccionar a los padres con el torneo anteriormente explicado. Para adaptarlo al estacionario, solo se hace la parte (\*\*) donde se aplica el cruce aritmético, y a la función en vez de pasarle todos los padres le pasamos el padre1 y padre2 y haciendo el cruce solamente sobre estos dos, generamos dos hijos, hijo1 e hijo2 que se devuelven por referencia.

```

1 funcion cruceAritmetico(problema, padres, referencia hijos, tam_sol):
2   para i <- 0 hasta TAM_POBLACION - 1 paso 2 hacer
3     hijo1 <- padres[i]
4     hijo2 <- padres[i+1]
5     // Cruzamos los NUM_CRUCES_ESPERADO primeros
6     si i < 2 * NUM_CRUCES_ESPERADO entonces
7       /**
8       // Aplicar cruce aritmetico
9       para j <- 0 hasta tam_sol - 1 hacer
10        r <- aleatorio(0.0, 1.0)
11        hijo1[j] <- r * padres[i][j] + (1 - r) * padres[i+1][j]
12        hijo2[j] <- r * padres[i+1][j] + (1 - r) * padres[i][j]
13      fin para
14      // Reparar hijos
15      fix(problema, hijo1)
16      fix(problema, hijo2)
17    fin si
18    hijos[i] <- hijo1
19    hijos[i+1] <- hijo2
20  fin para
21 fin funcion

```

### 2.6.2. $BLX - \alpha$

En el cruce  $BLX - \alpha$ , dadas dos soluciones padres,  $S_1$  y  $S_2$ , el algoritmo genera cada elemento de las soluciones hijas de forma independiente. Para cada empresa  $i$  seleccionamos el valor máximo y el mínimo entre los dos padres, y con estos nos definimos un intervalo, con los límites  $hmin_i$  y  $hmax_i$ , que se calculan de la siguiente forma:

$$hmin_i = \min(S_{i1}, S_{i2}) - \alpha \cdot I_i, \quad hmax_i = \max(S_{i1}, S_{i2}) + \alpha \cdot I_i \quad \forall i \in \{0, \dots, N-1\}$$

donde  $I_i = |S_{i1} - S_{i2}|$

Como nos pueden salir valores negativos, lo solucionamos de forma que nos quedamos con el máximo entre 0 y el valor resultante. También controlamos que puede suceder que todos los números seleccionados fueran 0, entonces lo que hacemos es quedarnos con el padre, aunque esto no me ha sucedido en ninguna ejecución. Este es el código que he usado para el algoritmo generacional, para adaptarlo al estacionario, solo haríamos la parte (\*\*) donde se aplica el cruce BLX, y a la función en vez de pasarle todos los padres le pasamos el padre1 y padre2 y haciendo el cruce sobre estos dos, nos genera dos hijos, hijo1 e hijo2, no un vector de hijos, que se devuelven por referencia.

```
1 funcion cruceBLX(problema, padres, referencia hijos, tam_sol):
2   para i <- 0 hasta TAM_POBLACION - 1 paso 2 hacer
3     hijo1 <- padres[i]
4     hijo2 <- padres[i+1]
5     todos_cero_hijo1 <- verdadero
6     todos_cero_hijo2 <- verdadero
7     si i < 2 * NUM_CRUCES_ESPERADO entonces
8       //(**)
9       para j <- 0 hasta tam_sol - 1 hacer
10        cmin <- min(padres[i][j], padres[i+1][j])
11        cmax <- max(padres[i][j], padres[i+1][j])
12        I <- cmax - cmin
13        lo_blx <- cmin - I * ALPHA
14        hi_blx <- cmax + I * ALPHA
15        // Nos quedamos con los maximos entre 0 y el valor para que no sea
16        // negativo
17        hijo1[j] <- max(aleatorio(lo_blx, hi_blx), 0.0)
18        si hijo1[j] != 0.0 entonces
19          todos_cero_hijo1 <- falso
20        fin si
21        hijo2[j] <- max(aleatorio(lo_blx, hi_blx), 0.0)
22        si hijo2[j] != 0.0 entonces
23          todos_cero_hijo2 <- falso
24        fin si
25      fin para
26      // (Aunque nunca sucede) Si hemos rellenado todo de ceros entonces
27      // mantenemos el padre para que no de un error al reparar
28      si todos_cero_hijo1 entonces hijo1 <- padres[i] fin si
29      si todos_cero_hijo2 entonces hijo2 <- padres[i+1] fin si
30      fix(problema, hijo1)
31      fix(problema, hijo2)
32    fin si
33    hijos[i] <- hijo1
34    hijos[i+1] <- hijo2
35  fin para
36 fin funcion
```

## 2.7. Operadores de mutación

La mutación de un individuo consiste en seleccionar de forma aleatoria dos empresas  $i$  y  $j$ , y de forma similar a como se cambia en la Búsqueda Local intercambiar un porcentaje máximo llamado ratio entre ambas, de forma que el resultado siga siendo una solución válida. Este porcentaje es el siguiente:

En la posición  $i$  el nuevo valor pasa a ser  $(1 - ratio) \cdot w_i$ , y en la posición  $j$  el nuevo valor pasa a ser  $w_j + ratio \cdot w_i$ . El máximo valor que tomará la variable será  $ratio = 0,15$

La mutación solo se realizará en el generacional un número esperado de mutaciones, sacando aleatoriamente los índices y en el estacionario sacando un número aleatorio y viendo si es menor o mayor que la probabilidad de mutar. Este pseudocódigo muestra como se ha implementado en el generacional, pero para el estacionario es similar solo que se lo hacemos solo a un hijo si el numero aleatorio sacado entre 0 y 1 es menor o igual que la probabilidad de mutación, y si hay que mutar se hace como en (\*\*) solo que para un solo hijo. Es interesante destacar que en el generacional relleno un vector de booleanos llamado *indices\_mutados* donde es true si un índice se ha mutado

y false en otro caso. Lo hago porque a la hora de actualizar los fitness de los hijos en el generacional, si un hijo no se ha mutado y no se ha cruzado entonces no tengo que volver a calcular su fitness, ya que es igual que el padre.

```

1  funcion mutar(referencia hijos, tam_sol, lo, hi, referencia indices_mutados):
2      indices_mutados <- todo falso
3      para i <- 0 hasta NUM_MUTACIONES_ESPERADO - 1 hacer
4          indice_hijo <- aleatorio(0, TAM_POBLACION - 1)
5          /**
6          indice_emp_1 <- aleatorio(0, tam_sol - 1)
7          indice_emp_2 <- aleatorio(0, tam_sol - 1)
8          si indice_emp_1 != indice_emp_2 entonces
9              valor_i <- hijos[indice_hijo][indice_emp_1]
10             valor_j <- hijos[indice_hijo][indice_emp_2]
11             si valor_i > lo y valor_j < hi entonces
12                 tope_1 <- 1 - (lo / valor_i)
13                 tope_2 <- (hi - valor_j) / valor_i
14                 tope_3 <- RATIO
15                 ratio <- min(tope_1, tope_2, tope_3)
16
17             si ratio == 0 entonces
18                 continuar
19             fin si
20             hijos[indice_hijo][indice_emp_1] <- (1 - ratio) * valor_i
21             hijos[indice_hijo][indice_emp_2] <- valor_j + ratio * valor_i
22             indices_mutados[indice_hijo] <- verdadero
23         fin si
24     fin para
25 fin funcion

```

## 2.8. Actualizar Fitness

Para los generacionales y los meméticos se usa una clase llamada Actualizador que se encarga de actualizar el valor del fitness de los hijos. Lo hace de forma que si un índice se ha cruzado o ha sido modificado entonces si hay que llamar a fitness y sumar una evaluación pero si no se ha cruzado ni modificado no es necesario, ya que sigue siendo igual que el padre. Esta función también se aprovecha de forma que devuelve el índice del mejor y peor fitness de esos hijos, que luego se usarán en el esquema de reemplazo:

```

1  funcion actualizarFitness(problema, fitness_padres, hijos, referencia fitness_hijos,
2      referencia indice_mejor_fitness, referencia indice_peor_fitness, evaluaciones):
3      mejor_fitness_local <- -infinito
4      peor_fitness_local <- infinito
5      indice_mejor <- 0, indice_peor <- 0
6      para i <- 0 hasta TAM_POBLACION - 1 hacer
7          // Evaluar solo los hijos cruzados o mutados
8          si i < 2 * NUM_CRUCES_ESPERADO o indices_mutados[i] entonces
9              fitness_hijos[i] <- fitness(hijos[i])
10             evaluaciones <- evaluaciones + 1
11         sino
12             // El hijo no fue cruzado ni mutado, hereda fitness del padre
13             fitness_hijos[i] <- fitness_padres[i]
14         fin si
15         si fitness_hijos[i] > mejor_fitness_local entonces // Actualizar mejor
16             mejor_fitness_local <- fitness_hijos[i]
17             indice_mejor <- i
18         fin si
19         si fitness_hijos[i] < peor_fitness_local entonces // Actualizar peor
20             peor_fitness_local <- fitness_hijos[i]
21             indice_peor <- i
22         fin si
23     fin para
24     indice_mejor_fitness <- indice_mejor
25     indice_peor_fitness <- indice_peor
26 fin funcion

```

### 3. Descripción de los algoritmos

#### 3.1. Algoritmos Genéticos

Para estos algoritmos se usa un tamaño de población igual 100, una probabilidad de cruce de 0.8 en generacionales y de 1 en estacionarios y una probabilidad de mutación de 0.1. El criterio de parada consistirá en realizar 10000 evaluaciones de la función objetivo.

##### 3.1.1. AGG-ARIT: Algoritmo Genético Generacional (con elitismo) con operador de cruce aritmético

El algoritmo consiste en sacar 100 padres según el torneo y cruzarlos usando el cruce aritmético en todos los padres, generando dos hijos cada dos padres, después mutar los hijos necesarios aleatoriamente, y por último evaluar los hijos que han sido mutados y/o cruzados. Una vez aquí si el mejor hijo no es mejor que el mejor de la población anterior (es decir el mejor global), entonces sustituimos el peor hijo por el mejor global. Después de todo esto hacemos el reemplazamiento, los hijos pasan a ser la nueva población.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0, mejor_fitness_global <- -infinito, mejor_global <- vacio
3      poblacion <- reservar(TAM_POBLACION), fitness_poblacion <- reservar(TAM_POBLACION)
4      // Inicializacion de la poblacion
5      para i <- 0 hasta TAM_POBLACION - 1 hacer
6          poblacion[i] <- crearSolucion(problema)
7          fitness_poblacion[i] <- fitness(poblacion[i])
8          evaluaciones <- evaluaciones + 1
9          si fitness_poblacion[i] > mejor_fitness_global entonces
10             mejor_global <- poblacion[i]
11             mejor_fitness_global <- fitness_poblacion[i]
12      fin si
13  fin para
14  tam_sol <- problema.obtenerTamanoSolucion()
15  rango <- problema.obtenerRangoDominio()
16  lo <- rango.inferior, hi <- rango.superior
17  // Inicializacion de las estructuras de la poblacion con tamano TAM_POBLACION
18  hijos <- reservar(TAM_POBLACION)
19  fitness_hijos <- reservar(TAM_POBLACION)
20  padres <- reservar(TAM_POBLACION)
21  fitness_padres <- reservar(TAM_POBLACION)
22  indices_mutados <- reservar(TAM_POBLACION)
23  actualizador <- crearActualizador(NUM_CRUCES_ESPERADO, TAM_POBLACION)
24
25  mientras evaluaciones < max_evals hacer
26      // Fase evolutiva
27      seleccionTorneo(poblacion, fitness_poblacion, padres, fitness_padres)
28      cruceAritmetico(problema, padres, hijos, tam_sol)
29      mutar(hijos, tam_sol, lo, hi, indices_mutados)
30      indice_mejor <- 0, indice_peor <- 0
31      // Actualizamos fitness hijos y devolvemos mejor y peor indice segun fitness
32      actualizador.actualizarFitness(problema, fitness_padres, hijos, fitness_hijos,
33          indice_mejor, indice_peor, indices_mutados, evaluaciones)
34      // Reemplazamiento
35      si fitness_hijos[indice_mejor] < mejor_fitness_global entonces
36          hijos[indice_peor] <- mejor_global
37          fitness_hijos[indice_peor] <- mejor_fitness_global
38          indice_mejor <- indice_peor
39      sino
40          mejor_global <- hijos[indice_mejor]
41          mejor_fitness_global <- fitness_hijos[indice_mejor]
42      fin si
43      poblacion <- hijos
44      fitness_poblacion <- fitness_hijos
45  fin mientras
46
47  retornar (mejor_global, mejor_fitness_global, evaluaciones)
48 fin funcion
```



### 3.1.2. AGG-BLX: Algoritmo Genético Generacional (con elitismo) con operador de cruce BLX- $\alpha$

Tomamos un valor de  $\alpha = 3$ . El algoritmo consiste en sacar 100 padres según el torneo y cruzarlos usando el cruce BLX en todos los padres, generando dos hijos por cada dos padres, después mutar los hijos necesarios aleatoriamente, y por último evaluar los hijos que han sido mutados y/o cruzados. Una vez aquí si el mejor hijo no es mejor que el mejor de la población anterior (es decir el mejor global), entonces sustituimos el peor hijo por el mejor global. Después de todo esto hacemos el reemplazamiento, los hijos pasan a ser la nueva población.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0, mejor_fitness_global <- -infinito, mejor_global <- vacio
3      poblacion <- reservar(TAM_POBLACION), fitness_poblacion <- reservar(TAM_POBLACION)
4      // Inicializacion de la poblacion
5      para i <- 0 hasta TAM_POBLACION - 1 hacer
6          poblacion[i] <- crearSolucion(problema)
7          fitness_poblacion[i] <- fitness(poblacion[i])
8          evaluaciones <- evaluaciones + 1
9          si fitness_poblacion[i] > mejor_fitness_global entonces
10             mejor_global <- poblacion[i]
11             mejor_fitness_global <- fitness_poblacion[i]
12      fin si
13  fin para
14  tam_sol <- problema.obtenerTamanoSolucion()
15  rango <- problema.obtenerRangoDominio()
16  lo <- rango.inferior, hi <- rango.superior
17  // Inicializacion de las estructuras de la poblacion con tamano TAM_POBLACION
18  hijos <- reservar(TAM_POBLACION)
19  fitness_hijos <- reservar(TAM_POBLACION)
20  padres <- reservar(TAM_POBLACION)
21  fitness_padres <- reservar(TAM_POBLACION)
22  indices_mutados <- reservar(TAM_POBLACION)
23  actualizador <- crearActualizador(NUM_CRUCES_ESPERADO, TAM_POBLACION)
24
25  mientras evaluaciones < max_evals hacer
26      // Fase evolutiva
27      seleccionTorneo(poblacion, fitness_poblacion, padres, fitness_padres)
28      cruceBLX(problema, padres, hijos, tam_sol)
29      mutar(hijos, tam_sol, lo, hi, indices_mutados)
30      indice_mejor <- 0, indice_peor <- 0
31      // Actualizamos fitness hijos y devolvemos mejor y peor indice segun fitness
32      actualizador.actualizarFitness(problema, fitness_padres, hijos, fitness_hijos,
33          indice_mejor, indice_peor, indices_mutados, evaluaciones)
34      // Reemplazamiento
35      si fitness_hijos[indice_mejor] < mejor_fitness_global entonces
36          hijos[indice_peor] <- mejor_global
37          fitness_hijos[indice_peor] <- mejor_fitness_global
38          indice_mejor <- indice_peor
39      sino
40          mejor_global <- hijos[indice_mejor]
41          mejor_fitness_global <- fitness_hijos[indice_mejor]
42      fin si
43      poblacion <- hijos
44      fitness_poblacion <- fitness_hijos
45  fin mientras
46
47  retornar (mejor_global, mejor_fitness_global, evaluaciones)
48 fin funcion
```

### 3.1.3. AGE-ARIT: Algoritmo Genético Estacionario con operador de cruce aritmético

El algoritmo consiste en sacar 2 padres según el torneo y cruzarlos usando el cruce aritmético, generando así dos hijos, después si es necesario (según una probabilidad) se mutan los dos hijos, y por último evaluar los dos hijos. Una vez aquí los dos hijos sustituyen a los dos peores de la población actual, en caso de ser mejores que ellos.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0, mejor_fitness_global <- -infinito, mejor_global <- vacio
3      poblacion <- reservar(TAM_POBLACION), fitness_poblacion <- reservar(TAM_POBLACION)
4
5      // Inicializacion de la poblacion
6      para i <- 0 hasta TAM_POBLACION - 1 hacer
7          poblacion[i] <- crearSolucion(problema)
8          fitness_poblacion[i] <- fitness(poblacion[i])
9          evaluaciones <- evaluaciones + 1
10         si fitness_poblacion[i] > mejor_fitness_global entonces
11             mejor_global <- poblacion[i]
12             mejor_fitness_global <- fitness_poblacion[i]
13         fin si
14     fin para
15
16     tam_sol <- problema.obtenerTamanoSolucion()
17     rango <- problema.obtenerRangoDominio()
18     lo <- rango.inferior, hi <- rango.superior
19
20     mientras evaluaciones < max_evals hacer
21         // Seleccion de dos padres mediante torneo
22         pos_1 <- seleccionTorneo(poblacion, fitness_poblacion)
23         pos_2 <- seleccionTorneo(poblacion, fitness_poblacion)
24
25         hijo1 <- crearSolucionVacua(tam_sol), hijo2 <- crearSolucionVacua(tam_sol)
26         cruceAritmetico(problema, poblacion[pos_1], poblacion[pos_2], tam_sol, hijo1,
27             hijo2)
28         mutar(problema, hijo1, tam_sol, lo, hi)
29         mutar(problema, hijo2, tam_sol, lo, hi)
30         f1 <- fitness(hijo1)
31         f2 <- fitness(hijo2)
32         evaluaciones <- evaluaciones + 2
33         peor1 <- 0, peor2 <- 0
34         // Metodo que devuelve en peor1 el peor y en peor2 el segundo peor
35         seleccionaDosPeores(fitness_poblacion, peor1, peor2)
36
37         // Reemplazamiento de los peores de la poblacion si los hijos los superan
38         si f1 > fitness_poblacion[peor1] entonces
39             fitness_poblacion[peor1] <- f1
40             poblacion[peor1] <- hijo1
41         fin si
42         si f2 > fitness_poblacion[peor2] entonces
43             fitness_poblacion[peor2] <- f2
44             poblacion[peor2] <- hijo2
45         fin si
46     fin mientras
47
48     indice_mejor <- 0
49     // Metodo que devuelve en indice_mejor el que mejor fitness tiene, lo llamamos aqui
50     // para devolver el mejor global
51     seleccionaMejor(fitness_poblacion, indice_mejor)
52     retornar (poblacion[indice_mejor], fitness_poblacion[indice_mejor], evaluaciones)
53 fin funcion
```

### 3.1.4. AGE-BLX: Algoritmo Genético Estacionario con operador de cruce BLX- $\alpha$

Tomamos un valor de  $\alpha = 3$ . El algoritmo consiste en sacar 2 padres según el torneo y cruzarlos usando el cruce BLX, generando así dos hijos, después si es necesario (según una probabilidad) se mutan los dos hijos, y por último evaluar los dos hijos. Una vez aquí los dos hijos sustituyen a los dos peores de la población actual, en caso de ser mejores que ellos.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0, mejor_fitness_global <- -infinito, mejor_global <- vacio
3      poblacion <- reservar(TAM_POBLACION), fitness_poblacion <- reservar(TAM_POBLACION)
4
5      // Inicializacion de la poblacion
6      para i <- 0 hasta TAM_POBLACION - 1 hacer
7          poblacion[i] <- crearSolucion(problema)
8          fitness_poblacion[i] <- fitness(poblacion[i])
9          evaluaciones <- evaluaciones + 1
10         si fitness_poblacion[i] > mejor_fitness_global entonces
11             mejor_global <- poblacion[i]
12             mejor_fitness_global <- fitness_poblacion[i]
13         fin si
14     fin para
15
16     tam_sol <- problema.obtenerTamanioSolucion()
17     rango <- problema.obtenerRangoDominio()
18     lo <- rango.inferior, hi <- rango.superior
19
20     mientras evaluaciones < max_evals hacer
21         // Seleccion de dos padres mediante torneo
22         pos_1 <- seleccionTorneo(poblacion, fitness_poblacion)
23         pos_2 <- seleccionTorneo(poblacion, fitness_poblacion)
24
25         hijo1 <- crearSolucionVacua(tam_sol), hijo2 <- crearSolucionVacua(tam_sol)
26         cruceBLX(problema, poblacion[pos_1], poblacion[pos_2], tam_sol, hijo1, hijo2)
27         mutar(problema, hijo1, tam_sol, lo, hi)
28         mutar(problema, hijo2, tam_sol, lo, hi)
29         f1 <- fitness(hijo1)
30         f2 <- fitness(hijo2)
31         evaluaciones <- evaluaciones + 2
32         peor1 <- 0, peor2 <- 0
33         // Metodo que devuelve en peor1 el peor y en peor2 el segundo peor
34         seleccionaDosPeores(fitness_poblacion, peor1, peor2)
35
36         // Reemplazamiento de los peores de la poblacion si los hijos los superan
37         si f1 > fitness_poblacion[peor1] entonces
38             fitness_poblacion[peor1] <- f1
39             poblacion[peor1] <- hijo1
40         fin si
41         si f2 > fitness_poblacion[peor2] entonces
42             fitness_poblacion[peor2] <- f2
43             poblacion[peor2] <- hijo2
44         fin si
45     fin mientras
46
47     indice_mejor <- 0
48     // Metodo que devuelve en indice_mejor el que mejor fitness tiene, lo llamamos aqui
49     // para devolver el mejor global
50     seleccionaMejor(fitness_poblacion, indice_mejor)
51
52     retornar (poblacion[indice_mejor], fitness_poblacion[indice_mejor], evaluaciones)
53 fin funcion
```

## 3.2. Algoritmos Meméticos

Los algoritmos meméticos resultan de hibridar el AGG que mejor resultado nos ha proporcionado anteriormente, que ha sido el que usa el cruce  $BLX - \alpha$  con la Búsqueda Local. Las probabilidades de cruce y mutación serán 0,8 (por cromosoma) y  $0,1/\text{número de genes}$  (por gen) en ambos casos, como en los AGG. El criterio de parada consistirá en realizar 10000 evaluaciones de la función objetivo, incluyendo las de la BL. Se aplicará la BL cada 10 generaciones del algoritmo, y cada vez que se aplique la BL se harán un máximo de 100 evaluaciones (si no converge antes de las 100 evaluaciones), y un ratio de 0.15.

### 3.2.1. AM-All

Cada 10 generaciones, se aplica la BL sobre todos los cromosomas de la población.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0, mejor_fitness_global <- -infinito, mejor_global <- vacio
3      poblacion <- reservar(TAM_POBLACION), fitness_poblacion <- reservar(TAM_POBLACION)
4
5      // Inicializacion poblacion
6      para i <- 0 hasta TAM_POBLACION - 1 hacer
7          poblacion[i] <- crearSolucion(problema)
8          fitness_poblacion[i] <- fitness(poblacion[i])
9          evaluaciones <- evaluaciones + 1
10         si fitness_poblacion[i] > mejor_fitness_global entonces
11             mejor_global <- poblacion[i]
12             mejor_fitness_global <- fitness_poblacion[i]
13         fin si
14     fin para
15
16     tam_sol <- problema.obtenerTamanoSolucion()
17     rango <- problema.obtenerRangoDominio()
18     lo <- rango.inferior, hi <- rango.superior
19     hijos <- reservar(TAM_POBLACION), fitness_hijos <- reservar(TAM_POBLACION)
20     padres <- reservar(TAM_POBLACION), fitness_padres <- reservar(TAM_POBLACION)
21     indices_mutados <- reservar(TAM_POBLACION)
22     contador_generaciones <- 0
23     // Incializamos el vector con todas las parejas de posiciones para BL
24     inicializarPosiciones(tam_sol)
25     tam_pos <- tam_sol * tam_sol
26
27     // Instancia de clase Búsqueda Local
28     bl <- crearBusquedaLocalAuxiliar(lo, hi, RATIO, MAX_EVALS_BL, max_evals)
29     actualizador <- crearActualizador(NUM_CRUCES_ESPERADO, TAM_POBLACION)
30
31     mientras evaluaciones < max_evals hacer
32         si contador_generaciones < 10 entonces
33             contador_generaciones <- contador_generaciones + 1
34             // Seleccion padres por torneo
35             seleccionTorneo(poblacion, fitness_poblacion, padres, fitness_padres)
36             cruceBLX(problema, padres, hijos, tam_sol)
37             mutar(hijos, tam_sol, lo, hi, indices_mutados)
38             indice_mejor <- 0, indice_peor <- 0
39
40             actualizador.actualizarFitness(problema, fitness_padres, hijos,
41                 fitness_hijos, indice_mejor, indice_peor, indices_mutados, evaluaciones
42                 )
43             // Mantenemos el mejor de la poblacion anterior si no hemos mejorado
44             si fitness_hijos[indice_mejor] < mejor_fitness_global entonces
45                 hijos[indice_peor] <- mejor_global
46                 fitness_hijos[indice_peor] <- mejor_fitness_global
47                 indice_mejor <- indice_peor // El mejor es ahora el que era el peor
48             sino // Tenemos un nuevo mejor global
49                 mejor_global <- hijos[indice_mejor]
50                 mejor_fitness_global <- fitness_hijos[indice_mejor]
51             fin si
52             poblacion <- hijos
53             fitness_poblacion <- fitness_hijos
54         sino
```

```

53     contador_generaciones <- 0
54     // Fase de optimizacion: Busqueda Local aplicada a toda la poblacion
55     para t <- 0 hasta TAM_POBLACION - 1 hacer
56         si evaluaciones < max_evals entonces
57             bl.aplicarBL(problema, poblacion[t], fitness_poblacion[t],
58                         posiciones, tam_pos, evaluaciones)
59         fin si
60     fin para
61     // Metodo que devuelve el indice del hijo con mejor fitness
62     indice_mejor <- seleccionaMejor(fitness_poblacion)
63     // Actualizar el mejor global
64     si fitness_poblacion[indice_mejor] > mejor_fitness_global entonces
65         mejor_global <- poblacion[indice_mejor]
66         mejor_fitness_global <- fitness_poblacion[indice_mejor]
67     fin si
68     fin mientras
69
70     retornar (mejor_global, mejor_fitness_global, evaluaciones)
71 fin funcion

```

Aquí se le aplica BL a todas las soluciones así que se eligen todas

### 3.2.2. AM-Rand

Cada 10 generaciones, se aplica la BL sobre un subconjunto de cromosomas de la población seleccionado aleatoriamente con probabilidad  $P_{LS} = 0,1$  para cada cromosoma.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0, mejor_fitness_global <- -infinito, mejor_global <- vacio
3      poblacion <- reservar(TAM_POBLACION), fitness_poblacion <- reservar(TAM_POBLACION)
4
5      // Inicializacion poblacion
6      para i <- 0 hasta TAM_POBLACION - 1 hacer
7          poblacion[i] <- crearSolucion(problema)
8          fitness_poblacion[i] <- fitness(poblacion[i])
9          evaluaciones <- evaluaciones + 1
10         si fitness_poblacion[i] > mejor_fitness_global entonces
11             mejor_global <- poblacion[i]
12             mejor_fitness_global <- fitness_poblacion[i]
13         fin si
14     fin para
15
16     tam_sol <- problema.obtenerTamanoSolucion()
17     rango <- problema.obtenerRangoDominio()
18     lo <- rango.inferior, hi <- rango.superior
19     hijos <- reservar(TAM_POBLACION), fitness_hijos <- reservar(TAM_POBLACION)
20     padres <- reservar(TAM_POBLACION), fitness_padres <- reservar(TAM_POBLACION)
21     indices_mutados <- reservar(TAM_POBLACION)
22     contador_generaciones <- 0
23     // Incializamos el vector con todas las parejas de posiciones para BL
24     inicializarPosiciones(tam_sol)
25     tam_pos <- tam_sol * tam_sol
26
27     // Instancia de clase Busqueda Local
28     bl <- crearBusquedaLocalAuxiliar(lo, hi, RATIO, MAX_EVALS_BL, max_evals)
29     actualizador <- crearActualizador(NUM_CRUCES_ESPERADO, TAM_POBLACION)
30
31     indices_aleatorios <- reservar(N_ALEATORIOS)
32     // Inicializamos las posiciones para luego seleccionar N aleatorias
33     inicializarPosicionesAleatorio(TAM_POBLACION)
34
35     mientras evaluaciones < max_evals hacer
36         si contador_generaciones < 10 entonces
37             contador_generaciones <- contador_generaciones + 1
38             // Seleccion padres por torneo
39             seleccionTorneo(poblacion, fitness_poblacion, padres, fitness_padres)
40             cruceBLX(problema, padres, hijos, tam_sol)
41             mutar(hijos, tam_sol, lo, hi, indices_mutados)
42             indice_mejor <- 0, indice_peor <- 0
43
44             actualizador.actualizarFitness(problema, fitness_padres, hijos,
45                 fitness_hijos, indice_mejor, indice_peor, indices_mutados, evaluaciones
46                 )
47             // Mantenemos el mejor de la poblacion anterior si no hemos mejorado
48             si fitness_hijos[indice_mejor] < mejor_fitness_global entonces
49                 hijos[indice_peor] <- mejor_global
50                 fitness_hijos[indice_peor] <- mejor_fitness_global
51                 indice_mejor <- indice_peor // El mejor es ahora el que era el peor
52             sino // Tenemos un nuevo mejor global
53                 mejor_global <- hijos[indice_mejor]
54                 mejor_fitness_global <- fitness_hijos[indice_mejor]
55             fin si
56             poblacion <- hijos
57             fitness_poblacion <- fitness_hijos
58         sino
59             contador_generaciones <- 0
60             // Metodo que selecciona N_Aleatorios (Se muestra pseudocodigo a
61                 continuacion)
62             seleccionaNAleatorios(poblacion, fitness_poblacion, indices_aleatorios)
63             para t <- 0 hasta N_ALEATORIOS - 1 hacer
```

```

61         si evaluaciones < max_evals entonces
62             bl.aplicarBL(problema, poblacion[indices_aleatorios[t]],
                fitness_poblacion[indices_aleatorios[t]], posiciones, tam_pos,
                evaluaciones)
63         fin si
64     fin para
65
66     indice_mejor <- seleccionaMejor(fitness_poblacion)
67
68     // Actualizar el mejor global
69     si fitness_poblacion[indice_mejor] > mejor_fitness_global entonces
70         mejor_global <- poblacion[indice_mejor]
71         mejor_fitness_global <- fitness_poblacion[indice_mejor]
72     fin si
73     fin si
74     fin mientras
75
76     retornar (mejor_global, mejor_fitness_global, evaluaciones)
77 fin funcion

```

Lo primero es aclarar que nos quedamos con  $N$  de la población, donde  $N = P_{LS} \cdot TAM\_POBLACION$ . Para seleccionar de forma aleatoria a  $N$  de la población sin repetir lo que hacemos es barajar un vector que contiene las posiciones desde  $[0, \dots, TAMANO\ POBLACION - 1]$  de forma que nos quedaremos con las  $N$  primeras de ese vector después de barajarlo.

```

1 funcion seleccionaNAleatorios(poblacion, fitness_pob, referencia indices_aleatorios):
2     // Barajamos el vector de posiciones aleatorias (va de 0 hasta TAM_POB-1)
3     barajar(posiciones_aleatorias)
4     indices_aleatorios.redimensionar(N_ALEATORIOS)
5     para i <- 0 hasta N_ALEATORIOS - 1 hacer
6         indices_aleatorios[i] <- posiciones_aleatorias[i] // guardamos la posicion
7         original
8     fin para
9 fin funcion

```

### 3.2.3. AM-Best

Cada 10 generaciones, aplicar la BL sobre los  $0,1 \cdot N$  mejores cromosomas de la población actual (N es el tamaño de la población).

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0, mejor_fitness_global <- -infinito, mejor_global <- vacio
3      poblacion <- reservar(TAM_POBLACION), fitness_poblacion <- reservar(TAM_POBLACION)
4
5      // Inicializacion poblacion
6      para i <- 0 hasta TAM_POBLACION - 1 hacer
7          poblacion[i] <- crearSolucion(problema)
8          fitness_poblacion[i] <- fitness(poblacion[i])
9          evaluaciones <- evaluaciones + 1
10         si fitness_poblacion[i] > mejor_fitness_global entonces
11             mejor_global <- poblacion[i]
12             mejor_fitness_global <- fitness_poblacion[i]
13         fin si
14     fin para
15
16     tam_sol <- problema.obtenerTamanoSolucion()
17     rango <- problema.obtenerRangoDominio()
18     lo <- rango.inferior, hi <- rango.superior
19     hijos <- reservar(TAM_POBLACION), fitness_hijos <- reservar(TAM_POBLACION)
20     padres <- reservar(TAM_POBLACION), fitness_padres <- reservar(TAM_POBLACION)
21     indices_mutados <- reservar(TAM_POBLACION)
22     contador_generaciones <- 0
23     // Incializamos el vector con todas las parejas de posiciones para BL
24     inicializarPosiciones(tam_sol)
25     tam_pos <- tam_sol * tam_sol
26
27     // Instancia de clase Busqueda Local
28     bl <- crearBusquedaLocalAuxiliar(lo, hi, RATIO, MAX_EVALS_BL, max_evals)
29     actualizador <- crearActualizador(NUM_CRUCES_ESPERADO, TAM_POBLACION)
30
31     indices_mejores <- reservar(N_MEJORES)
32
33     mientras evaluaciones < max_evals hacer
34         si contador_generaciones < 10 entonces
35             contador_generaciones <- contador_generaciones + 1
36             // Seleccion padres por torneo
37             seleccionTorneo(poblacion, fitness_poblacion, padres, fitness_padres)
38             cruceBLX(problema, padres, hijos, tam_sol)
39             mutar(hijos, tam_sol, lo, hi, indices_mutados)
40             indice_mejor <- 0, indice_peor <- 0
41
42             actualizador.actualizarFitness(problema, fitness_padres, hijos,
43                 fitness_hijos, indice_mejor, indice_peor, indices_mutados, evaluaciones
44                 )
45             // Mantenemos el mejor de la poblacion anterior si no hemos mejorado
46             si fitness_hijos[indice_mejor] < mejor_fitness_global entonces
47                 hijos[indice_peor] <- mejor_global
48                 fitness_hijos[indice_peor] <- mejor_fitness_global
49                 indice_mejor <- indice_peor // El mejor es ahora el que era el peor
50             sino // Tenemos un nuevo mejor global
51                 mejor_global <- hijos[indice_mejor]
52                 mejor_fitness_global <- fitness_hijos[indice_mejor]
53             fin si
54             poblacion <- hijos
55             fitness_poblacion <- fitness_hijos
56         sino
57             contador_generaciones <- 0
58             // Metodo que selecciona los N_Mejores (Se muestra pseudocodigo a
59             // continuacion)
60             seleccionaNMejores(poblacion, fitness_poblacion, indices_mejores)
61             para t <- 0 hasta N_MEJORES - 1 hacer
62                 si evaluaciones < max_evals entonces
63                     bl.aplicarBL(problema, poblacion[indices_mejores[t]],
```



```

        fitness_poblacion[indices_mejores[t]], posiciones, tam_pos,
        evaluaciones)
61     fin si
62   fin para
63
64   indice_mejor <- seleccionaMejor(fitness_poblacion)
65
66   // Actualizar el mejor global
67   si fitness_poblacion[indice_mejor] > mejor_fitness_global entonces
68     mejor_global <- poblacion[indice_mejor]
69     mejor_fitness_global <- fitness_poblacion[indice_mejor]
70   fin si
71   fin si
72 fin mientras
73
74 retornar (mejor_global, mejor_fitness_global, evaluaciones)
75 fin funcion

```

Para seleccionar los N mejores lo que hacemos es ordenar de forma descendente según su fitness a los índices de la población, y nos quedamos con los N primeros índices de ese orden descendente, que son los N mejores porque son los que tienen un fitness mayor.

```

1 funcion seleccionaNMejores(poblacion, fitness_pob, referencia indices_mejores):
2   indices <- reservar(TAM_POBLACION)
3   Rellenar indices con valores secuenciales desde 0 hasta TAM_POBLACION - 1
4   // Ordenar indices segun el fitness de mayor a menor
5   ordenar(indices, fitness_pob, descendente)
6   indices_mejores.redimensionar(N_MEJORES)
7   para i <- 0 hasta N_MEJORES - 1 hacer
8     indices_mejores[i] <- indices[i] // guardamos la posicion original
9   fin para
10 fin funcion

```

### 3.2.4. Uso de Búsqueda Local (BL)

En esta práctica usaremos la Búsqueda Local en los algoritmos meméticos, y el código será común a los tres. Se ha modificado un poco respecto a la práctica anterior, de forma que ahora tenemos una clase llamada *BusquedaLocalAuxiliar* a la que en el constructor le pasamos el número máximo de evaluaciones para la búsqueda local, el máximo de evaluaciones globales (para no pasarnos) y los valores de *lo*, *hi* y *ratio*. A la función de búsqueda le pasamos como argumento una solución, su fitness, el vector de posiciones por referencia, su tamaño, y el número de evaluaciones globales que lleva, también por referencia para ir aumentándolo desde aquí. El algoritmo de búsqueda local consiste en primero barajar el vector de posiciones y después importante para no pasarnos, definir el número máximo de evaluaciones que va a hacer, lo definimos como el mínimo entre el máximo que tiene permitido y las que le quedan para llegar al máximo global, después se aplica una búsqueda local tipo primero mejor, explorando un vecindario formado por todos los pares de posiciones  $(i, j) \forall i, j \in 0, 1, \dots, N$ , donde nos movemos a la primera solución mejor. Para cada par, se genera un vecino transfiriendo un porcentaje del peso desde la posición *i* a la posición *j*. Este porcentaje es el siguiente:

En la posición *i* el nuevo valor pasa a ser  $(1 - ratio) \cdot w_i$ , y en la posición *j* el nuevo valor pasa a ser  $w_j + ratio \cdot w_i$ .

El máximo valor que tomará la variable será *ratio* = 0,15

El pseudocódigo se muestra en la siguiente página.

```

1  funcion aplicarBL(problem, sol, fit, posiciones, tam_pos, evaluaciones):
2      barajar(posiciones)
3      pos <- 0
4      maximo_evaluaciones_BL <- mininimo(evaluaciones + max_evals_BL, max_evals_global)
5      mientras pos < tam_pos y evaluaciones < maximo_evaluaciones_BL hacer
6          pos_i <- posiciones[pos].first
7          pos_j <- posiciones[pos].second
8          si pos_i != pos_j entonces
9              valor_anterior_i <- sol[pos_i]
10             valor_anterior_j <- sol[pos_j]
11             si valor_anterior_i > lo y valor_anterior_j < hi entonces
12                 tope_1 <- 1.0 - (lo / valor_anterior_i)
13                 tope_2 <- (hi - valor_anterior_j) / valor_anterior_i
14                 tope_3 <- ratio_max
15                 // Nos quedamos con el menor tope y ese es lo maximo que podemos
16                 intercambiar
17                 ratio <- min({tope_1, tope_2, tope_3})
18                 si ratio == 0 entonces
19                     pos <- pos + 1
20                     continuar
21                 fin si
22                 sol[pos_i] <- (1 - ratio) * valor_anterior_i
23                 sol[pos_j] <- valor_anterior_j + ratio * valor_anterior_i
24
25             si problem.isValid(sol) entonces // hay que hacerlo porque por
26                 redondeo a veces se quedan < lo
27                 new_fit <- problem.fitness(sol)
28                 evaluaciones <- evaluaciones + 1
29                 si new_fit > fit entonces // Encontrado el primero mejor
30                     fit <- new_fit
31                     Random.shuffle(posiciones)
32                     pos <- 0
33                 sino // Recuperamos la solucion como era porque el cambio o no es
34                     valido o no mejora
35                     sol[pos_i] <- valor_anterior_i
36                     sol[pos_j] <- valor_anterior_j
37                     pos <- pos + 1
38                 fin si
39             sino
40                 sol[pos_i] <- valor_anterior_i
41                 sol[pos_j] <- valor_anterior_j
42                 pos <- pos + 1
43             fin si
44         sino
45             pos <- pos + 1
46         fin si
47     fin mientras
48 fin funcion

```

## 4. Estructura del código y manual de usuario

### 4.1. Estructura del código

Esta es la estructura general del proyecto, donde en la carpeta build están los archivos generados automáticamente durante la compilación del proyecto, en configuraciones están las configuraciones de cada algoritmo, en datos-portfolio-2526 los csv con los datos de las bolsas, en inc los archivos .h y en src los .cpp, en common los archivos comunes a todos, y en resultados las gráficas y el script para generarlas.

```
|-- build
|-- CMakeLists.txt
|-- common
|-- configuraciones
|-- datos_portfolio_2526
|-- ejecutar_todo.sh
|-- inc
|-- resultados
|-- LICENSE
|-- main.cpp
|-- README.md
|-- src
```

### 4.2. Compilación

Nos situamos en la carpeta software y hacemos lo siguiente:

- cd build
- rm -rf \*
- cmake ..
- make
- cd ..

### 4.3. Ejecución

Ahora después de hacer la compilación anterior nos situamos en la carpeta software y hacemos:

- ./ejecutar\_todo.sh semilla\_inicial
- Ejemplo de uso ./ejecutar\_todo.sh 1 empezará con la semilla 1 y terminará con 50 la última ejecución

### 4.4. Gráficas

Nos situamos en la carpeta software de nuevo:

- cd resultados
- python3 graficas.py

## 5. Experimentos y análisis de resultados

### 5.1. Descripción de los experimentos

Se utilizan datos bursátiles reales (2015-2025) de empresas con presencia continua en las siguientes bolsas de valores:

- `ibex_35_data.csv`: IBEX 35 con 30 empresas y 2813 días. Parámetros ( $lo = 0,005, hi = 0,08$ ).
- `syp_100_data.csv`: S&P 100 con 97 empresas y 2765 días. Parámetros ( $lo = 0,005, hi = 0,05$ ).
- `syp_500_data.csv`: S&P 500 con 457 empresas y 2765 días. Parámetros ( $lo = 0,005, hi = 0,02$ ).

Para que la comparación sea justa se utilizará un máximo de 10000 evaluaciones, una evaluación es cada vez que llamamos a la función *fitness*

- $maxevals = 10000$

Se va a utilizar como semilla desde 1, sumando 1 en cada iteración hasta llegar a 50 incluido.

### 5.2. Resultados

Resultados medios de los algoritmos en cada bolsa:

| Algoritmo | Fitness (2015–2024) | Fitness (2025) | Beneficio (2025) | Evaluaciones | Tiempo (s) |
|-----------|---------------------|----------------|------------------|--------------|------------|
| Greedy    | -4.5584             | -3.2545        | 0.0401           | 1.0000       | 0.0007     |
| RS        | -5.0948             | -3.3365        | 0.0490           | 10000.0000   | 0.0271     |
| BL        | -4.5794             | -3.1105        | 0.0214           | 1201.5600    | 0.0027     |
| AGG-ARIT  | -4.8859             | -3.2067        | 0.0486           | 10037.5400   | 0.0255     |
| AGG-BLX   | -4.6324             | -3.1334        | 0.0359           | 10053.6600   | 0.0264     |
| AGE-ARIT  | -4.7521             | -3.1551        | 0.0434           | 10000.0000   | 0.0250     |
| AGE-BLX   | -4.6224             | -3.1348        | 0.0341           | 10000.0000   | 0.0274     |
| AM-ALL    | -4.8053             | -3.1730        | 0.0445           | 10000.0000   | 0.0486     |
| AM-RAND   | -4.5857             | -3.1225        | 0.0284           | 10003.0200   | 0.0347     |
| AM-BEST   | -4.5687             | -3.1191        | 0.0275           | 10002.0200   | 0.0338     |

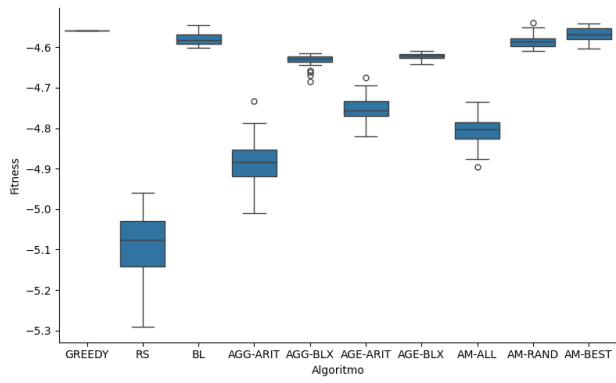
Cuadro 2: Ibex35 resultados (medias)

| Algoritmo | Fitness (2015–2024) | Fitness (2025) | Beneficio (2025) | Evaluaciones | Tiempo (s) |
|-----------|---------------------|----------------|------------------|--------------|------------|
| Greedy    | -2.8350             | -3.6912        | 0.0929           | 1.0000       | 0.0014     |
| RS        | -3.4369             | -4.1258        | 0.0784           | 10000.0000   | 0.0977     |
| BL        | -3.0298             | -3.8724        | 0.0978           | 5229.4600    | 0.0753     |
| AGG-ARIT  | -3.1805             | -3.8782        | 0.0811           | 10026.0200   | 0.1106     |
| AGG-BLX   | -2.9520             | -3.7464        | 0.0936           | 10042.3200   | 0.1021     |
| AGE-ARIT  | -3.1595             | -3.8676        | 0.0831           | 10000.0000   | 0.1045     |
| AGE-BLX   | -2.9336             | -3.7208        | 0.0961           | 10000.0000   | 0.0929     |
| AM-ALL    | -3.2624             | -3.9474        | 0.0806           | 10000.0000   | 0.4274     |
| AM-RAND   | -2.9648             | -3.7559        | 0.0925           | 10000.0000   | 0.2679     |
| AM-BEST   | -2.9092             | -3.7428        | 0.0950           | 10000.0000   | 0.2577     |

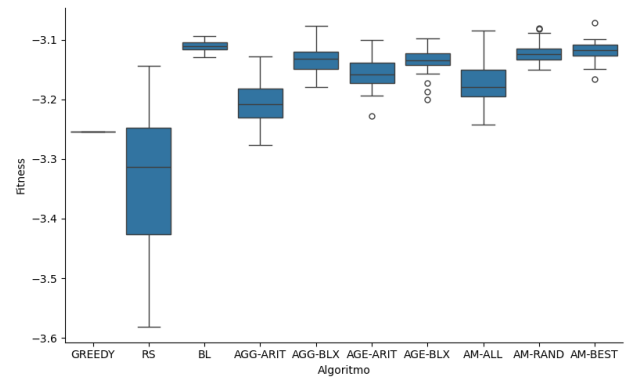
Cuadro 3: SyP100 resultados (medias)

| Algoritmo | Fitness (2015–2024) | Fitness (2025) | Beneficio (2025) | Evaluaciones | Tiempo (s) |
|-----------|---------------------|----------------|------------------|--------------|------------|
| Greedy    | -2.7495             | -3.7768        | 0.0134           | 1.0000       | 0.0070     |
| RS        | -3.7173             | -4.4645        | 0.0144           | 10000.0000   | 0.4326     |
| BL        | -3.2789             | -4.2016        | 0.0074           | 10000.0000   | 0.9508     |
| AGG-ARIT  | -3.0531             | -4.0035        | 0.0164           | 10049.0000   | 0.4286     |
| AGG-BLX   | -2.8810             | -3.9307        | 0.0198           | 10055.5000   | 0.4449     |
| AGE-ARIT  | -3.0018             | -3.9704        | 0.0185           | 10000.0000   | 0.4137     |
| AGE-BLX   | -2.7495             | -3.8857        | 0.0225           | 10000.0000   | 0.4312     |
| AM-ALL    | -3.4446             | -4.2367        | 0.0176           | 10000.0000   | 5.8740     |
| AM-RAND   | -3.0387             | -4.0164        | 0.0169           | 10000.0000   | 3.4892     |
| AM-BEST   | -2.8946             | -3.9627        | 0.0153           | 10000.0000   | 3.4688     |

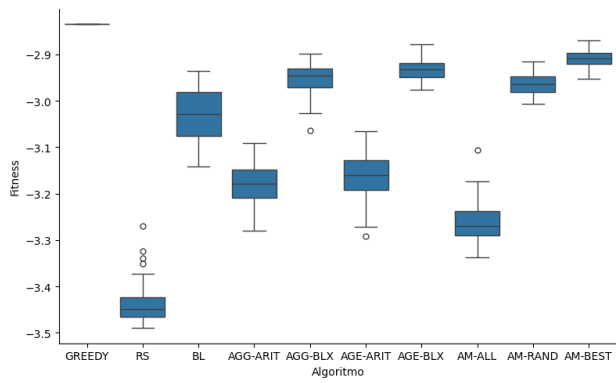
Cuadro 4: Resultados (medias)



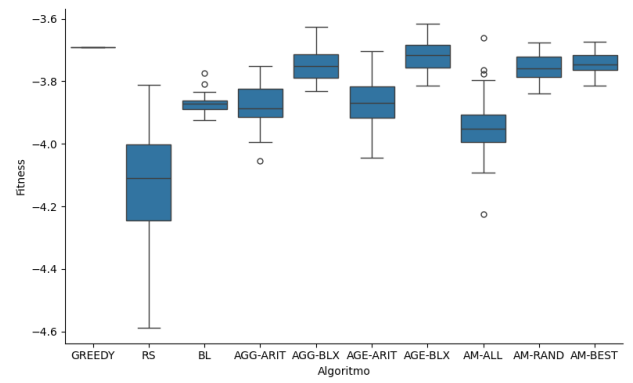
(a) IBEX35 2015-2024



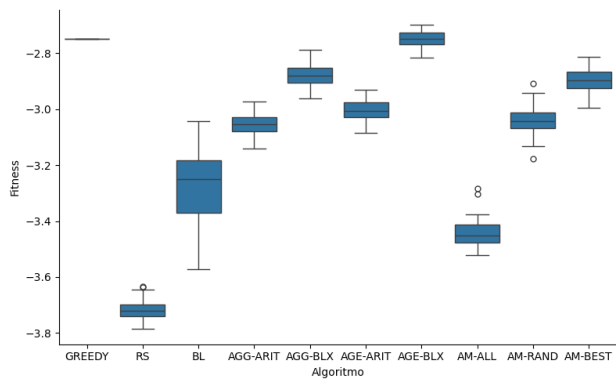
(b) IBEX35 2025



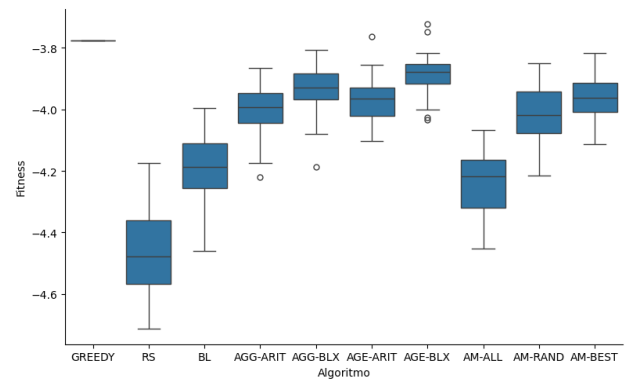
(c) SyP100 2015-2024



(d) SyP100 2025



(e) SyP500 2015-2024



(f) SyP500 2025

Figura 1: Conjunto de resultados

### 5.3. Análisis de resultados

Para poder hacer un análisis mejor, primero vamos a mostrar como se ordenan los algoritmos de menor a mayor ranking medio. La media de rankings de cada algoritmo según su fitness histórico:

Cuadro 5: Ranking y ranking medio de los algoritmos en cada dataset.

| Algoritmo | IBEX35 | SyP 100 | SyP 500 | Ranking Medio |
|-----------|--------|---------|---------|---------------|
| Greedy    | 1.0    | 1.0     | 1.5     | <b>1.17</b>   |
| AM-BEST   | 2.0    | 2.0     | 4.0     | <b>2.67</b>   |
| AGE-BLX   | 5.0    | 3.0     | 1.5     | <b>3.17</b>   |
| AGG-BLX   | 6.0    | 4.0     | 3.0     | <b>4.33</b>   |
| AM-RAND   | 4.0    | 5.0     | 6.0     | <b>5.00</b>   |
| BL        | 3.0    | 6.0     | 8.0     | <b>5.67</b>   |
| AGE-ARIT  | 7.0    | 7.0     | 5.0     | <b>6.33</b>   |
| AGG-ARIT  | 9.0    | 8.0     | 7.0     | <b>8.00</b>   |
| AM-ALL    | 8.0    | 9.0     | 9.0     | <b>8.67</b>   |
| RS        | 10.0   | 10.0    | 10.0    | <b>10.00</b>  |

Lo primero que podemos deducir de los resultados es que en este problema el cruce BLX es mejor que el aritmético, tanto en estacionarios como en generacionales. Esto se debe a que en el cruce aritmético el hijo siempre queda entre  $\min(\text{padre1}[j], \text{padre2}[j])$  y  $\max(\text{padre1}[j], \text{padre2}[j])$ , es decir, nunca sale del rango de los padres. El cruce BLX en cambio extiende ese rango con el parámetro  $\alpha$  haciendo que el hijo puede quedar fuera del rango de los padres, explorando zonas del espacio que el aritmético nunca alcanzaría. Esto hace que en el aritmético la diversidad caiga rápidamente y el algoritmo se estanque en óptimos locales haciendo que los hijos casi no mejoren o ya no mejoren en cada generación. Sin embargo, BLX mantiene más diversidad durante más generaciones gracias a poder explorar una zona del espacio mayor. Por eso lo he usado luego en los meméticos.

En los algoritmos genéticos, nos da un mejor resultado los estacionarios que los generacionales, el AGE reemplaza solo dos individuos por iteración, lo que genera:

- Mejor preservación de buenas soluciones ya que el AGG toda la población se reemplaza cada generación, lo que significa que soluciones buenas pueden perderse aunque haya elitismo, podemos tener una muy buena y 99 muy malas, y eso pasaría a la siguiente generación. Sin embargo, el AGE solo se reemplazan los peores si los hijos los mejoran, por lo que las buenas soluciones permanecen más tiempo en la población.
- Convergencia más controlada. Al cambiar solo dos individuos por iteración, el AGE avanza más despacio pero de forma más precisa hacia buenas regiones del espacio. En cambio el AGG al renovar toda la población puede ir saltando fuera de buenas regiones en una sola generación.

Esto conecta con lo comentado anteriormente entre los cruces BLX y aritmético. El AGE mantiene diversidad de forma natural al ir reemplazando los individuos poco a poco, por lo que no necesita tanto del poder exploratorio de BLX para evitar convergencia prematura, sin embargo AGG si que es más dependiente del operador de cruce para mantener la diversidad. Por eso en los resultados obtenidos la diferencia entre BLX y aritmético es menor en AGE que en AGG.

Los resultados de los algoritmos meméticos muestran que el peor es AM-All, esto se debe a que como cada 10 generaciones hace búsqueda local sobre toda la población, consume muchas evaluaciones en BL, lo que hace que no queden evaluaciones para hacer muchas generaciones, y además aplica BL sobre soluciones que pueden no ser buenas. De hecho posiblemente solo haga las 10 primeras generaciones, y luego aplique BL y ni siquiera termine de aplicar BL a todos los de la población, ya que tienen vectores de cambio de posiciones con tamaños de  $30 \times 30 = 900$ ,  $97 \times 97 = 9.400$  y  $457 \times 457 = 208.849$  por lo que las 100 evaluaciones de la búsqueda local las gastan siempre ya nunca recorren el vector de posiciones entero. Esto hace que en BL al aplicárselo a toda la población ya se vayan  $100 \times 100 = 10.000$  evaluaciones, luego más las que lleva de las 10 generaciones anteriores, nunca hace más de diez generaciones.

AM-Best es el mejor de los tres porque es el que mejor combina el uso del generacional con la BL, ya que hace las 10 generaciones y luego explora localmente muchos menos que AM-All, por lo que le siguen quedando evaluaciones para seguir explorando más generaciones todavía, que es muy importante porque cada nueva generación introduce mayor diversidad, evitando así una convergencia prematura. Y AM-Rand esta entre medias de ambos porque también maneja de forma similar a AM-Best el numero de evaluaciones en BL, pero al usar N aleatorios no

aprovechamos esa búsqueda local, porque mejoramos individuos mediocres, en cambio, con AM-Best se intensifica la búsqueda en las regiones más prometedoras del espacio mientras el resto de la población sobre la que no se hace BL mantiene su diversidad y sigue explorando otras regiones.

Si comparamos un poco los resultados obtenidos con BL y los algoritmos meméticos podemos ver que la búsqueda local es bastante buena en el IBEX35 ya que al haber solo 30 empresas, la búsqueda local parte de una solución aleatoria que puede ir mejorando muchas veces recorriendo todo el vecindario si es necesario, de hecho termina en menos de 10.000 evaluaciones lo que indica que termina porque está en un óptimo local, luego no tiene vecinos mejores esa solución, pero sin embargo conforme hay más empresas la cosa empeora, ya que en el SyP500 tenemos un  $457 \times 457 = 208.849$  vecinos, lo que hace que solo pueda explorar una mínima parte del vecindario, por lo que depende mucho de la solución que partimos inicialmente, ya que es muy difícil que esté en una región prometedora. AM-Best supera a BL porque su fase evolutiva le permite explorar múltiples regiones del espacio simultáneamente mediante una población diversa, y la BL luego mejora las mejores soluciones encontradas. Esto es especialmente ventajoso en SyP100 y SyP500 porque al tener esa diversidad podemos explorar más regiones y no solo una. Sin embargo AM-ALL queda por debajo de la BL porque al aplicar BL masivamente nos estamos quedando con lo peor de ambos algoritmos, ni explora bien como un algoritmo evolutivo ni mejora tan eficientemente como una BL dedicada solo a una solución.

Haciendo un análisis más general de todos los algoritmos de esta práctica, se puede observar que conforme mayor es el número de empresas, peor son los algoritmos meméticos y mejores los que usan el cruce BLX. Los meméticos van empeorando porque conforme aumenta el número de empresas, ya que la búsqueda local con solo 100 evaluaciones como máximo mejora poco o nada y estamos desperdiciando esas 100 evaluaciones por cada individuo al que le aplicamos búsqueda local. Por otra parte los que usan BLX va mejorando porque no desperdicia esas evaluaciones que los meméticos sí cuando hay muchas empresas. Por eso con pocas empresas como es pequeño el número de vecinos, los meméticos si que hacen una mejora real en la BL y no las desperdician esas evaluaciones. Sin embargo se puede ver que los aritméticos el que mejora más es el que usa AGE por que permite más diversidad como hemos comentado antes pero con AGG no mejora con más empresas porque converge prematuramente (tiene mejor ranking pero porque BL empeora mucho más).

Como hemos dicho antes, la búsqueda local depende mucho de la solución aleatoria con la que empezamos por eso en los boxplots su altura es mucho mayor que en la de los demás algoritmos de esta práctica (en especial se puede apreciar en el SyP500 ya que es muy importante la solución de la que partimos). Los de esta práctica tienen todos una dispersión de soluciones similar entre ellos, ya que como hemos dicho exploramos muchas más soluciones y por tanto acabamos llegando a resultados no tan diversos en distintas ejecuciones.

Aunque teóricamente en cuanto a tiempos de ejecución los algoritmos meméticos deberían de ser más rápidos que los genéticos, cuando aplicamos BL en los meméticos tenemos que barajar vectores muy grandes, por eso no se nota tanto en el IBEX y si que se nota mas en SyP100 y SyP500, de hecho en el SyP500 barajamos un vector de  $457 \times 457 = 208.849$  elementos lo que se lleva la inmensa mayoría del tiempo de ejecución. En la búsqueda local normal no tarda tanto porque al ser un vector tan grande recorre mucho más antes de barajar que en los meméticos, que al tener un máximo de 100 evaluaciones en BL, cada 100 evaluaciones lo barajamos. En definitiva que en los meméticos lo barajamos más veces que en BL normal.

En la siguiente página muestro algunos algoritmos extra.



## 6. Algoritmos extra

### 6.1. Extra 1: AM-BEST-MOD

Este algoritmo es igual a AM-Best solo que hemos cambiado el número máximo de evaluaciones de BL a 500. La idea es que según lo que hemos explicado antes debería de ser peor en SyP100 y SyP500 porque desperdicia más evaluaciones en la BL y debería de ser un poco mejor en IBEX35 porque su número de vecinos es de 900 y entonces en BL puede explorar localmente más. Los resultados obtenidos son los siguientes y confirman lo que queríamos demostrar:

| Algoritmo   | Fitness (2015–2024) | Evaluaciones | Tiempo (s) |
|-------------|---------------------|--------------|------------|
| AM-BEST     | -4.5687             | 10002.0200   | 0.0338     |
| AM-BEST-MOD | -4.5596             | 10000.0000   | 0.0328     |

Cuadro 6: Ibex35 resultados (medias)

| Algoritmo   | Fitness (2015–2024) | Evaluaciones | Tiempo (s) |
|-------------|---------------------|--------------|------------|
| AM-BEST     | -2.9092             | 10000.0000   | 0.2577     |
| AM-BEST-MOD | -2.9171             | 10000.0000   | 0.3545     |

Cuadro 7: SyP100 resultados (medias)

| Algoritmo   | Fitness (2015–2024) | Evaluaciones | Tiempo (s) |
|-------------|---------------------|--------------|------------|
| AM-BEST     | -2.8946             | 10000.0000   | 3.4688     |
| AM-BEST-MOD | -2.9891             | 10000.0000   | 6.3291     |

Cuadro 8: Resultados (medias)

### 6.2. Extra 2: AGE-BLX-MOD

Este algoritmo es parecido a AGE-BLX solo que ahora en el cruce se generan cuatro hijos y nos quedamos con los 4 mejores y nos olvidamos de la mutación. Los resultados obtenidos son los siguientes:

| Algoritmo   | Fitness (2015–2024) | Evaluaciones | Tiempo (s) |
|-------------|---------------------|--------------|------------|
| AGE-BLX     | -4.6224             | 10000.0000   | 0.0274     |
| AGE-BLX-MOD | -4.7455             | 10000.0000   | 0.0265     |

Cuadro 9: Ibex35 resultados (medias)

| Algoritmo   | Fitness (2015–2024) | Evaluaciones | Tiempo (s) |
|-------------|---------------------|--------------|------------|
| AGE-BLX     | -2.9336             | 10000.0000   | 0.0929     |
| AGE-BLX-MOD | -2.9217             | 10000.0000   | 0.0895     |

Cuadro 10: SyP100 resultados (medias)

| Algoritmo   | Fitness (2015–2024) | Evaluaciones | Tiempo (s) |
|-------------|---------------------|--------------|------------|
| AGE-BLX     | -2.7495             | 10000.0000   | 0.4312     |
| AGE-BLX-MOD | -2.7410             | 10000.0000   | 0.4148     |

Cuadro 11: Resultados (medias)

Es peor en el IBEX35 porque al haber solo 30 empresas no es difícil encontrar una región prometedora, sin embargo, en los otros dos al tener un número mucho mayor de empresas es más difícil encontrar regiones prometedoras, y quedándonos con los dos mejores hijos de 4 parece que mejoramos un poco.

## 7. Referencias

1. Guión de las prácticas y material de teoría de la asignatura.
2. OpenAI. (2026). *ChatGPT* (Versión GPT-4o) <https://chatgpt.com>
3. Google. (2026). *Gemini* (Versión 1.5 Pro) <https://gemini.google.com>